

LAB SESSION 7

Lab Name: Reliability, Dependability & Security Engineering

Project: MediScript-E — Digital Healthcare Platform

Objectives

- Identify system risks and failure scenarios in MediScript-E
- Design reliable and secure system components
- Perform threat analysis and define security controls

Theory

Dependability includes:

- **Reliability:** System performs correctly over time
- **Availability:** System is accessible when needed
- **Security:** System is protected from unauthorized access and misuse
- **Maintainability:** System can be updated and repaired efficiently

Security engineering protects systems from misuse and attacks. In a healthcare platform like MediScript-E, security is critical because sensitive medical data, personal information, and authentication credentials are involved.

Task 1: Failure Scenario Identification

Failure ID	Failure Scenario	Component Affected	Impact	Likelihood
FS-01	Database connection timeout	Prisma / Supabase PostgreSQL	High - All features fail	Medium
FS-02	Email service (SMTP) unavailable	Nodemailer / Gmail SMTP	High - Verification, OTP, reminders fail	Low
FS-03	Supabase Storage unavailable	Medical Vault	Medium - File upload/download fails	Low
FS-04	JWT secret key compromised	NextAuth session management	Critical - All sessions compromised	Very Low
FS-05	Vercel serverless function timeout	API Routes	High - API requests fail	Low
FS-06	Invalid/expired email verification token	Auth module	Medium - User cannot verify email	Medium

Failure ID	Failure Scenario	Component Affected	Impact	Likelihood
FS-07	OTP brute force attack	2FA module	High - Account takeover possible	Medium
FS-08	Unauthorized API access without session	All protected API routes	Critical - Data breach	Low
FS-09	File upload with malicious content	Medical Vault	High - Server compromise	Low
FS-10	Admin accidentally deletes critical user	Admin module	Medium - Data loss	Low
FS-11	Cron job fails to execute	Medicine reminder system	Low - Reminders not sent	Medium
FS-12	OAuth provider (Google/GitHub) downtime	Authentication module	Medium - OAuth login unavailable	Very Low
FS-13	SQL injection via user input	Database layer	Critical - Data breach	Low
FS-14	Session token theft (XSS)	Frontend / NextAuth	Critical - Account hijacking	Low
FS-15	Concurrent appointment booking conflict	Appointment module	Medium - Double booking	Medium
FS-16	Groq API unavailable	AI Chatbot (MediBot)	Low - Chatbot unavailable	Low
FS-17	AI chatbot providing harmful medical advice	MediBot	High - Patient harm	Low
FS-18	Search returning other users' data	Search module	Critical - Privacy breach	Low

Task 2: Reliability Targets

Component	Reliability Target	Measurement	Current Implementation
Overall System Uptime	99.9%	Monthly availability	Vercel production deployment
Database Availability	99.95%	Monthly uptime	Supabase managed PostgreSQL
API Response Time	< 2 seconds	95th percentile	Vercel serverless functions
Email Delivery	> 95% success rate	Per email sent	Gmail SMTP via Nodemailer

Component	Reliability Target	Measurement	Current Implementation
Authentication Success	> 99% for valid credentials	Per login attempt	NextAuth with bcrypt
File Upload Success	> 98%	Per upload attempt	Supabase Storage
OTP Delivery	> 95% within 60 seconds	Per OTP request	Gmail SMTP
Cron Job Execution	> 99% on schedule	Per scheduled run	External cron service
AI Chatbot Response	> 95% within 3 seconds	Per message	Groq API (Llama 3.1)

Reliability Design Decisions

Decision	Justification
Supabase managed PostgreSQL	Automatic backups, high availability, managed SSL
Vercel serverless deployment	Auto-scaling, zero-downtime deployments
Connection pooling (pg Pool, max: 20)	Prevents database connection exhaustion
JWT session strategy	Stateless, no server-side session storage needed
OTP expiry (10 minutes)	Balances security and usability
Email verification token expiry (24 hours)	Sufficient time for user action
Groq API with system prompt	Restricts AI to platform-related responses only

Task 3: Threat Analysis

3.1 Authentication Threats

Threat ID	Threat	Attack Vector	Likelihood	Impact
T-01	Brute force password attack	Repeated login attempts	Medium	High
T-02	Credential stuffing	Using leaked credentials from other sites	Medium	High
T-03	OTP brute force	Repeated OTP guesses	Medium	High
T-04	Session token theft	XSS attack stealing JWT	Low	Critical
T-05	OAuth token interception	Man-in-the-middle attack	Very Low	Critical
T-06	Email verification bypass	Manipulating verification URL	Low	High
T-07	2FA bypass via direct API call	Calling signIn with fake bypass params	Low	Critical

3.2 Data Threats

Threat ID	Threat	Attack Vector	Likelihood	Impact
T-08	SQL injection	Malicious input in forms	Low	Critical
T-09	Unauthorized data access	Missing auth checks on API routes	Low	Critical
T-10	Insecure file upload	Uploading malicious files	Low	High
T-11	Data exposure via API	API returning sensitive fields	Medium	High
T-12	CSRF attack	Forged cross-site requests	Low	High
T-13	Search returning cross-user data	Missing role-based search filters	Low	Critical

3.3 Infrastructure Threats

Threat ID	Threat	Attack Vector	Likelihood	Impact
T-14	Environment variable exposure	Leaked .env file in repository	Low	Critical
T-15	Hardcoded credentials in code	Developer mistake	Very Low	Critical
T-16	DDoS attack	Flooding API endpoints	Low	High
T-17	AI prompt injection	Malicious user input to chatbot	Low	Medium

Task 4: Security Controls

4.1 Authentication Security Controls

Threat	Security Control	Implementation in MediScript-E
T-01 Brute force	Password hashing with bcryptjs	<code>bcrypt.hash(password, 10)</code> — 10 salt rounds
T-02 Credential stuffing	Email verification requirement	<code>emailVerified: false</code> blocks login until verified
T-03 OTP brute force	OTP expiry + single use	OTP expires in 10 minutes, cleared after use
T-04 Session theft	JWT with secure secret	<code>NEXTAUTH_SECRET</code> environment variable, HttpOnly cookies
T-05 OAuth interception	HTTPS enforced	Vercel enforces HTTPS on all routes
T-06 Verification bypass	Token expiry + unique token	<code>verificationExpires</code> checked, token cleared after use

Threat	Security Control	Implementation in MediScript-E
T-07 2FA bypass	OTP null check before bypass	<code>twoFactorCode !== null</code> verified before session creation

4.2 Data Security Controls

Threat	Security Control	Implementation in MediScript-E
T-08 SQL injection	Prisma ORM parameterized queries	All DB operations via Prisma — no raw SQL
T-09 Unauthorized access	Role-based access control (RBAC)	<code>getServerSession()</code> checked on every protected API route
T-10 Malicious file upload	File type and size validation	Validated before Supabase Storage upload
T-11 Data exposure	Selective field queries	Prisma <code>select</code> used to return only required fields
T-12 CSRF	NextAuth CSRF protection	Built-in CSRF token in NextAuth
T-13 Cross-user search	Role-based search filtering	Search API filters by authenticated user's ID and role

4.3 Infrastructure Security Controls

Threat	Security Control	Implementation in MediScript-E
T-14 Env variable exposure	<code>.env</code> in <code>.gitignore</code>	<code>.env</code> never committed to repository
T-15 Hardcoded credentials	Environment variables only	All secrets in <code>.env</code> / Vercel environment variables
T-16 DDoS	Vercel rate limiting	Vercel platform-level DDoS protection
T-17 AI prompt injection	System prompt restrictions	MediBot system prompt restricts to platform topics only

Security Design Document

Security Layers

Layer 1: Network Security

└─ HTTPS enforced (Vercel)

└─ SSL/TLS database connection (`sslmode=no-verify`)

└─ Supabase Storage secure URLs

Layer 2: Authentication Security

└─ `bcryptjs` password hashing (10 salt rounds)

- └─ Email verification (24-hour token expiry)
- └─ JWT session management (30-day expiry)
- └─ OAuth via Google & GitHub (auto-verified, profile picture)
- └─ 2FA Email OTP (10-minute expiry, single use)

Layer 3: Authorization Security

- └─ Role-Based Access Control (PATIENT / DOCTOR / ADMIN)
- └─ getSession() on all protected API routes
- └─ Admin cannot delete own account
- └─ Patients can only access their own data
- └─ Search results filtered by user role and ownership

Layer 4: Input Security

- └─ Prisma ORM (prevents SQL injection)
- └─ Input validation on all API routes
- └─ File type and size validation for uploads
- └─ Email format validation on registration
- └─ AI chatbot system prompt prevents harmful responses

Layer 5: Data Security

- └─ Passwords never stored in plaintext
- └─ OTP cleared from database after use
- └─ Verification tokens cleared after use
- └─ Selective field queries (no sensitive data exposure)

RBAC Matrix

Action	Public	Patient	Doctor	Admin
View landing page	✓	✓	✓	✓
Use AI chatbot	✓	✓	✓	✓
Register / Login	✓	✓	✓	✓
Book appointment	×	✓	×	×
Manage appointments	×	×	✓	×
Issue prescription	×	×	✓	×
Archive/Edit/Delete prescription	×	×	✓	×
View prescription	×	✓	✓	×
Upload medical document	×	✓	×	×
Set medicine reminder	×	✓	×	×
View admin dashboard	×	×	×	✓
Delete users	×	×	×	✓
View all appointments	×	×	×	✓
Toggle 2FA	×	✓	✓	×

Action	Public	Patient	Doctor	Admin
Global search	×	✓	✓	✓

Key Findings / Learning Outcomes

- Identified **18 failure scenarios** and **17 security threats** specific to MediScript-E
- New threats added: AI prompt injection (T-17), cross-user search data (T-13), Groq API failure (FS-16)
- Understood that AI chatbot introduces new security considerations — system prompt must restrict harmful responses
- Recognized that search functionality must be role-based to prevent cross-user data exposure
- Learned that **Prisma ORM** inherently prevents SQL injection through parameterized queries
- Recognized that **2FA** significantly reduces account takeover risk even when passwords are compromised
- Designed a comprehensive **RBAC matrix** ensuring each role has access only to relevant features